

Implement Bersenham's line algorithm, Midpoint circle and Ellipse algorithm

Aim:

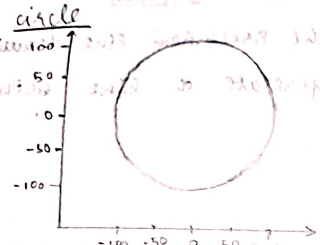
To implement the Bresenham line drawing algorithm in Python to generate a line between two given points.

Program:

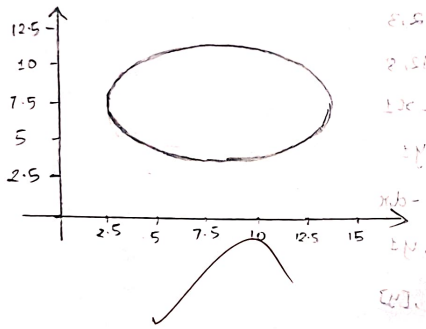
```
import matplotlib.pyplot as plt
x1, y1 = 2,3
x2, y2 = 12, 8
dx = x2 - x1
dy = y2 - y1
p = 2 * dy - dx
x, y = x1, y1
X, Y = [x], [y]
while x < x2:
    x += 1
    if p < 0:
        p += 2 * dy
    else:
        y += 1
        p += 2 * (dy - dx)
    X.append(x)
    Y.append(y)
plt.plot([X[0], X[-1]], [Y[0], Y[-1]], 'o-')
plt.grid()
plt.show()
```

Result: Thus the python program to generate a line using Bersenham's algorithm was successfully implemented.

choice 1: circle



choice 2: Eclipse



Aim:

To implement Bersenham's circle Algorithm & Elipse drawing algorithm in python & display the generated circle or elipse using matplotlib.

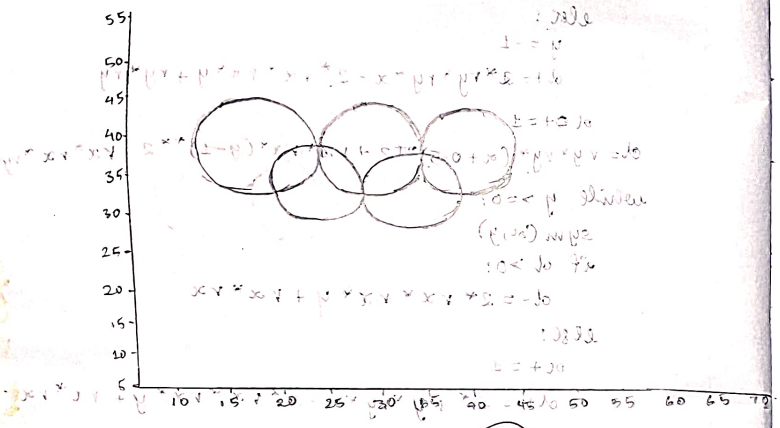
Program:

```
import matplot.pyplot as plt
p = []
def sym(x, y, e = false):
    pts = [(x,y), (-x,y), (x,-y), (-x,-y)]
    p.extend (pts + [(y, x), (-y,x), (y,-x), (-y,-x)] if
                                        e else[])
if int (input ("circle 2. elipse: ")) == 1:
    r = 100
    x, y, d = 0, r, 3 - 2*r
    while x <= y:
        sym (x, y, True)
        if (d < 0):
            d += 4* x + 6
        else:
            d += 4 * (x - y) + 10
            y -= 1
        x += 1
else:
    rx, ry = 120, 80
    x, y = 0, ry
```

```
d = ry * ry - rx * rx * ry + 0.25 * rx * rx
while 2*ry*ry*x <= 2*rx*rx*y:
    sym(x,y)
    if (d<0):
        d+= 2*ry*ry*x + ry*ry
    else:
        y = -1
        d+= 2*ry*ry*x - 2*rx*rx*y + ry*ry
    x+=1
d= ry*ry*(x+0.5)**2 + rx*rx*(y-1)**2 - rx*rx*ry*ry
while y >=0:
    sym(x,y)
    if d >0:
        d-= 2*rx*rx*y + rx*rx
    else:
        x+=1
        d+= 2*ry*ry*x - 2*rx*rx*y + rx*rx
x,y = zip(set(p))
plt.scatter(x,y, s=5, c='b')
plt.axis("equal")
plt.axis("off")
plt.show()
```

Result:

Thus the implementation of Bresenham's circle & ellipse drawing algorithms in python was successfully completed.



Aim:

To implement the midpoint algorithm in python & draw olympic logo.

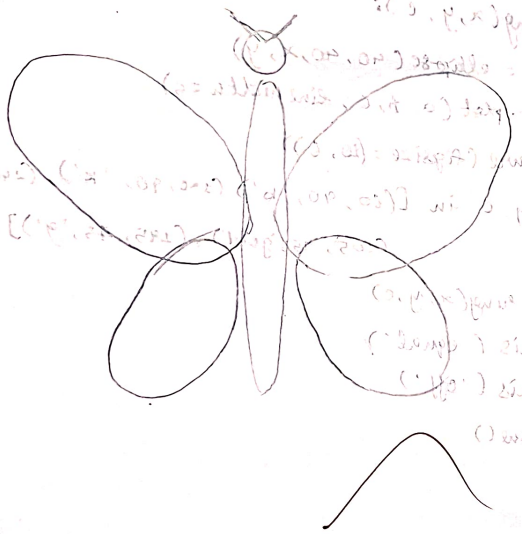
Program:

```
import matplotlib.pyplot as plt
def ellipse (rx, ry, xc, yc):
    p = [] ; x, y = 0, ry
    p = ry*ry - rx*ry + 0.25*rx*rx
    while 2*ry*ry*x < 2*rx*rx*y:
        p += [(xc+x, yc+y), (xc-x, yc+y),
              (xc-x, yc-y), (xc+x, yc-y)]
        if p<0:
            x+=1
            p+= 2*ry*ry*x + ry*ry
        else:
            x+=1; y-=1
            p+=2*ry*ry*x - 2*rx*rx*y + ry*ry
    p = ry*ry*(x+.5)**2 + rx*rx*(y-1)**2 -
                                rx*rx*ry*ry
    while y>=0:
        p += [(xc+x, yc+y), (xc-x) y(+y),
              (xc-x, yc-y), (xc+x, yc-y)]
        if p>0:
            y-=1; p-=2*rx*rx*y - rx*rx
```

```
        else:
            x1 = 1; y -= 1
            p += 2*ry*ry*x - 2*rx*rx*y + rx*rx + rx*rx
    return zip(*p)
def ring(x, y, e):
    a, b = ellipse(40, 40, x, y)
    plt.plot(a, b, c, linewidth=5)
plt.figure(figsize=(10, 6))
for x, y, e in [(60, 90, 'b'), (150, 90, 'k'), (240, 90, 'r'),
                (105, 45, 'gold'), (195, 45, 'g')]:
    ring(x, y, e)
plt.axis('equal')
plt.axis('off')
plt.show()
```

Result:

Thus the midpoint algorithm to draw olympic logo using ellipse in python has been implemented successfully.



Aim:

To draw butterfly using ellipse with midpoint algorithm.

Program:

```
import matplotlib.pyplot as plt
import numpy as np
def ellipse (cx,cy, rx, ry, angle, color):
    rx = abs (rx)
    ry = abs (ry)
    t = np. linspace (0, 2* np.pi, 300)
    r_coords = rx* np. cos(t)
    y_coords = ry * np. sin(t)
    xr = x-coords * np. cos (angle) - y-coords * np. sin(angle)
    yr = x-coords * np. sin(angle) + y-coords * np. cos(angle)
    plt. plot (xr+cx, yr +cy, color = color, linewidth=2)
ellipse (-3, 2, 2.5, 4, np. radians (30), 'blue')
ellipse (-3,-2, 2, 3, np. radians (-30), 'cyan')
ellipse (3, 2, 2.5, 4, np. radians (-30), 'red')
ellipse (3,-2, 2, 3, np. radians(30), 'magenta')
ellipse (0, 0, 0.5, 5, 0, 'black')
ellipse (0, 5.8, 0.7, 0.8, 0, 'black')
plt. plot ([0,-1], [6.2, 7.5], color = 'black', linewidth = 1.5)
plt. plot ([0, 1], [6.2, 7.5], color = 'black', linewidth = 1.
```

```
plt.axis('equal')
plt.axis('off')
plt.show()
```

Result:

Thus the butterfly using midpoint algorithm in python has been implemented successfully.

Aim:

To draw a smiling face using midpoint algorithm in python

Program:

```
import matplotlib.pyplot as plt
import numpy as np
def e(cx, cy, rx, ry, c):
    t = np.linspace(0, 2*np.pi, 100)
    plt.plot(cx + rx*np.cos(t), cy + rx*np.sin(t), c)

def smile(cx, cy, rx, ry):
    t = np.linspace(np.pi, 2*np.pi, 100)
    plt.plot(cx + rx*np.cos(t), cy + ry*np.sin(t), 'r')

plt.figure(figsize=(6, 6))
e(0, 0, 5, 5, 'orange')
e(-2, 1, 5, .5, .8, 'b')
e(2, 1.5, .5, .8, 'b')
e(0, 0, .4, .8, 'g')
smile(0, -1, 5, 2, 1)
plt.axis('equal')
plt.xlim(-6, 6)
plt.ylim(-6, 6)
plt.grid(True)
plt.show()
```

Result: Thus the smiling face using midpoint algorithm has been implemented successfully.

Aim:

To implement the midpoint ellipse drawing algorithm using python and draw horizontal ellipse then convert into vertical ellipse.

Program:

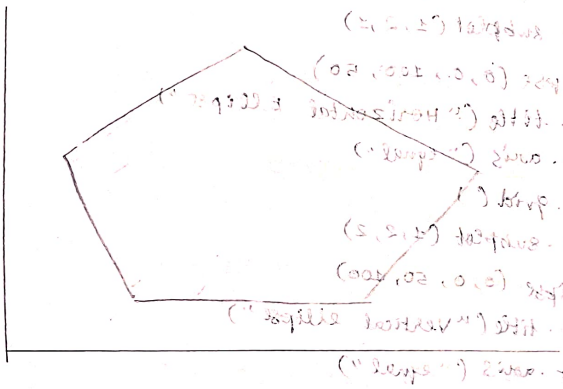
```
import matplotlib.pyplot as plt
def plot_points(cx, cy, x, y):
    for a, b in [(x, y), (-x, y), (x, -y), (-x, -y)]:
        plt.plot(cx+b, cy+b, 'o', markersize=1)

def ellipse(cx, cy, rx, ry):
    x, y = 0, ry
    rx2, ry2 = rx**2, ry**2
    while 2*ry2*x < 2*rx2*y:
        plot_points(cx, cy, x, y)
        if p < 0:
            x += 1
            p += 2*ry2*x + ry2
        else:
            x += 1
            y -= 1
            p += 2*ry2*x - 2*rx2*y + ry2
    p = ry2*(x+0.5)**2 + rx2*(y-1)**2 - rx2*ry2
    while y >= 0:
        plot_points(cx, cy, x, y)
        if p > 0:
            y -= 1
            p += rx2 - 2*rx2*y
```

```
        else:
            x += 1
            y -= 1
            p += 2*ry2*x - 2*rx2*y + rx2
    plt.figure(figsize=(10,5))
    plt.subplot(1,2,1)
    ellipse(0,0,100,50)
    plt.title("Horizontal Ellipse")
    plt.axis("equal")
    plt.grid()
    plt.subplot(1,2,2)
    ellipse(0,0,50,100)
    plt.title("Vertical ellipse")
    plt.axis("equal")
    plt.grid()
    plt.show()
```

Result:

The Horizontal & vertical ellipse were successfully drawn using the Midpoint Ellipse Algorithm.





Implement scan conversion of a polygon and use flood-fill algorithms to fill areas in a graphic windows

Aim:

Implement flood filling algorithm to change the color.

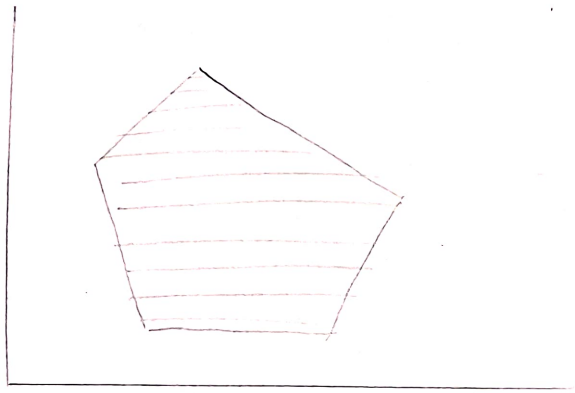
Program:

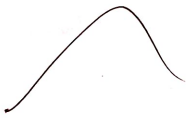
```
def flood_fill (image, x, y, old_color, new_color):
    if old_color == new color :
        return
    if x <0 or x>= len(image) or y <0 or y>=
                                    len(image[0]):
        return
    if image[x][y] != old_color:
        return.
    image[x][y] = new_color
    flood_fill (image, x-1, y, old_color, new_color)
    flood_fill (image, x+1, y, old_color, new_color)
    flood_fill (image, x, y-1, old_color, new_color)
    flood_fill (image, x, y+1, old_color, new_color)

image = [
        [0,0,0,0,0],
        [0,1,1,1,0],
        [0,1,1,1,0]
        [0,0,0,0,0]
        ]
flood_fill (image, 1,1,1,2)
```

Result:

Thus the flood-filling algorithm has been executed successfully.





Aim:

To implement midpoint algorithm to draw a polygon and scan-line for colouring

Program:

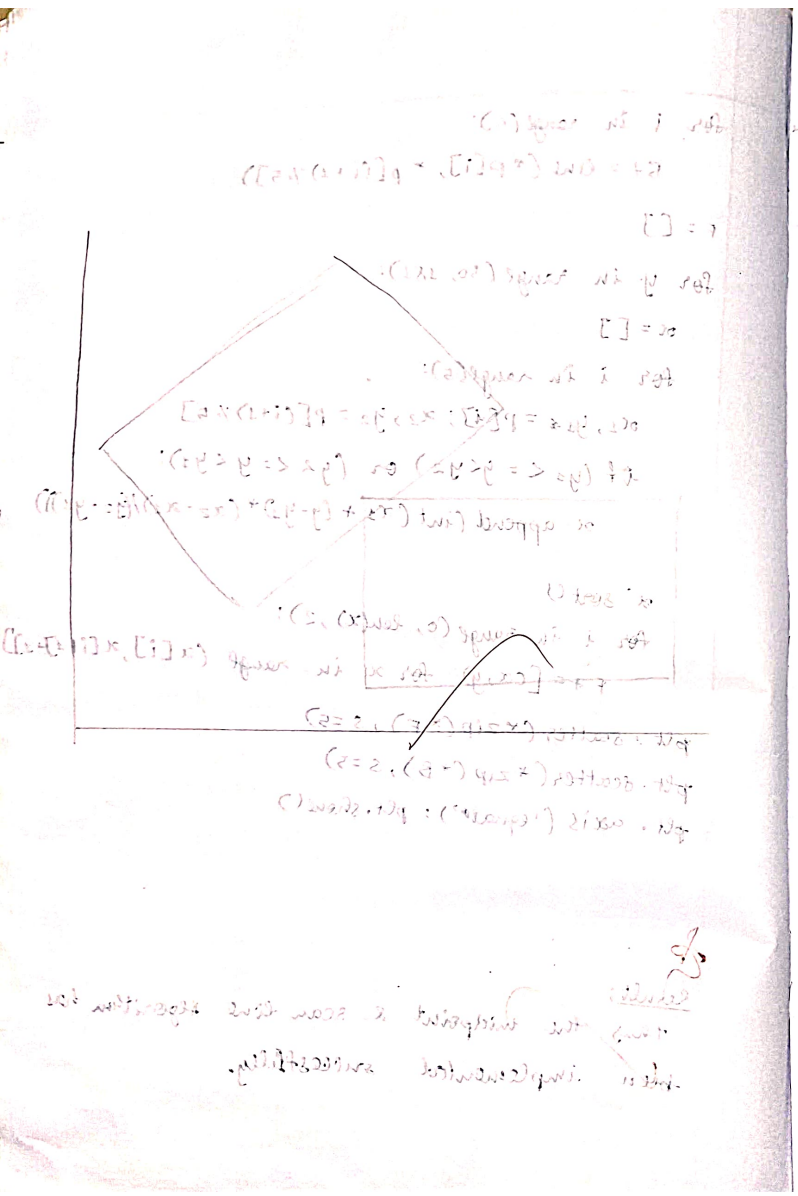
```
import matplotlib.pyplot as plt
def line(x1,y1,x2,y2):
    p=[]; dx=abs(x2-x1); dy=abs(y2-y1)
    sx=1 if x2>x1 else -1; sy=1 if y2>y1 else -1
    x,y=x1,y1
    if dx>dy:
        d=2*dy-dx
        for i in range(dx+1):
            p.append((x,y)); x+=sx
            if d>=0: y+=sy; d-=2*dy
            d+=2*dy
    else:
        d=2*dx-dy
        for i in range(dy+1):
            p.append((x,y)); y+=sy
            if d>=0: x+=sx; d-=2*dy
            d+=2*dx
    return p.
p=[(50,50),(150,50),(180,120),(100,180),(30,120)]
B=[]
```

```
for i in range(5):
    B+= line(*p[i], *p[(i+1)%5])
F = []
for y in range(50, 181):
    x = []
    for i in range(5):
        x1, y1 = p[i]; x2, y2 = p[(i+1)%5]
        if (y1 <= y<y2) or (y2 <= y < y1):
            x.append(int(x1 + (y-y1)*(x2-x1)/(y2-y1)))

    x.sort()
    for i in range(0, len(x), 2):
        F+= [(x, y) for x in range(x[i], x[i+1]+1)]
plt.scatter(*zip(*F), s=5)
plt.scatter(*zip(*B), s=5)
plt.axis('equal'); plt.show()
```

Result:

Thus the midpoint & scan line algorithm has been implemented successfully.



Ex. No.:
Date:

2D Transformations - Translation, Scaling and Rotation

Aim:

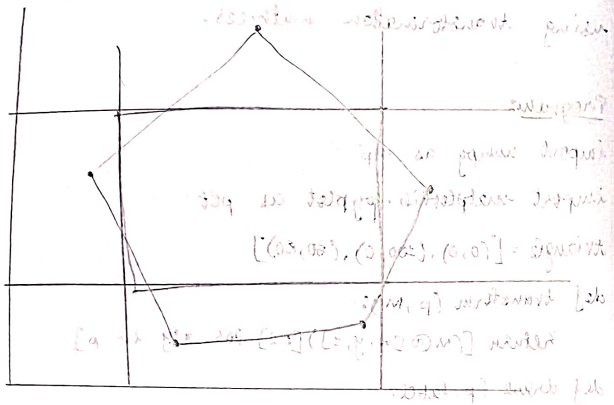
To write a program that performs translation, scaling Rotation on basic 2D shapes (e.g., triangle, rectangle) using transformation matrices.

Program:

```
import numpy as np
import matplotlib.pyplot as plt
triangle = [(0,0),(100,0),(50,80)]
def transform(p,M):
    return [(M@[x,y,1])[:2] for x,y in p]
def draw(p, label):
    p = p+[p[0]]
    x,y = zip(*p)
    plt.plot(x,y, label = label)
T = [[1,0,120],[0,1,50],[0,0,1]]
S = [[1.5,0,0],[0,1.5,0],[0,0,1]]
a = np.radians(45)
R = [[np.cos(a), -np.sin(a), 0],
     [np.sin(a), np.cos(a), 0],
     [0,0,1]]
```

Result:

The 2D transformations (translation, scaling, and rotation) were successfully applied using matrix operations on a triangle.



Ex. No.:
Date:

Polygon Clipping using Sutherland-Hodgman Algorithm

Aim:

To write program that clips a polygon to a specified rectangular window using the Sutherland-Hodgman Polygon Clipping Algorithm and displays the clipped polygon.

Program:

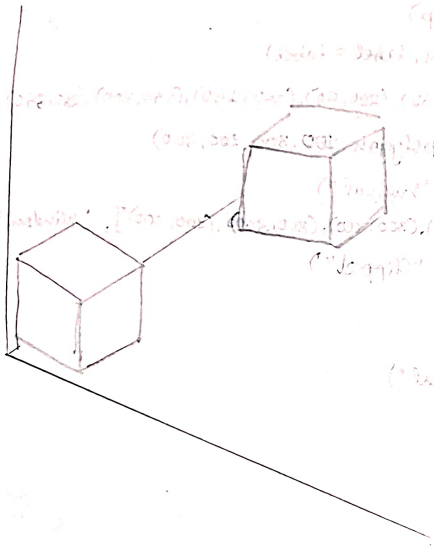
```
import matplotlib.pyplot as plt
def clip(poly, xmin, xmax, ymin, ymax):
    for edge in range(4):
        out = []
        s = poly[-1]
        for p in poly:
            x,y = p
            sx, sy = s
            inside = [x >= xmin, x <= xmax, y >= ymin, y <= ymax][edge]
            sinside = [sx >= xmin, sx <= xmax, sy >= ymin, sy <= ymax]
                                                              [edge]
            if inside != sinside:
                if edge < 2:
                    x = xmin if edge == 0 else xmax
                    y = sy + (y-sy) * (x-sx) / (x-sx)
                else:
                    y = ymin if edge == 2 else ymax
                    x = sx + (x - sx) * (y-sy) / (y - sy)
                out.append((x,y))
            if inside:
                out.append(p)

            s=p
        poly = out
    return poly
```

```
def draw(p, label):
    p += [p[0]]
    x, y = zip(*p)
    plt.plot(x, y, label = label)
polygon = [(50, 150), (200, 50), (350, 150), (350, 300), (250, 350), (150, 300)]
clipped = clip(polygon, 100, 300, 100, 300)
draw(polygon, "original")
draw([(100, 100), (300, 100), (300, 300), (100, 300)], "window")
draw(clipped, "Clipped")
plt.legend()
plt.grid()
plt.axis("equal")
plt.show()
```

Result:

The polygon was successfully clipped using the Sutherland-Hodgman algorithm against a rectangular clipping window.



Date:

3D Transformations on Basic Objects (Cube, Pyramid)

Aim:

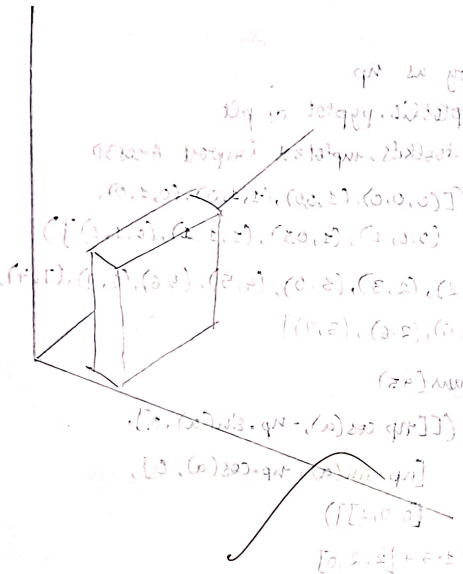
To write a program that allows the user to perform 3D transformations (translation, scaling, rotation) on basic 3D objects like a cube or pyramid, and visualize the results.

Program:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
v = np.array([[0,0,0],[1,0,0],[1,1,0],[0,1,0],
              [0,0,1],[1,0,1],[1,1,1],[0,1,1]])
e = [(0,1),(1,2),(2,3),(3,0),(4,5),(5,6),(6,7),(7,4),
     (0,4),(1,5),(2,6),(3,7)]
a = np.radians(45)
R = np.array([[np.cos(a),-np.sin(a),0],
              [np.sin(a),np.cos(a),0],
              [0,0,1]])
v2 = v @ R.T * 1.5 + [2,2,0]
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
for i,j in e:
    ax.plot(*zip(v[i],v[j]),'b')
    ax.plot(*zip(v2[i],v2[j]),'r')
ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.set_zlabel("Z")
plt.show()
```

Result:

The user was able to perform translation, scaling and rotation on a 3D cube. The transformed cube was successfully rendered and visualized.



Ex. No.:
Date

Create and Render 3D Objects using Polygons and Apply Basic Color and Shading Techniques

Aim:
To create and render 3D object (such as cube or sphere) using polygonal modeling and apply: Flat shading, Basic color techniques.

Program:

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
V = [(0,0,0),(1,0,0),(1,1,0),(0,1,0),
     (0,0,1),(1,0,1),(1,1,1),(0,1,1)]
f = [[V[i] for i in x] for x in
     [(0,1,2,3),(4,5,6,7),(0,1,5,4),
      (2,3,7,6),(1,2,6,5),(4,7,3,0)]]
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.add_collection3d(
    Poly3DCollection(f,
        facecolors=['red','blue','green','yellow','cyan','orange'],
        edgecolors='black'))
ax.set_box_aspect([1,1,1])
plt.show()
```

Result:
3D polygonal objects were successfully created and rendered. Basic color and flat shading techniques were applied to enhance visual representation.

Ex.No.:
Date:

Create a Multimedia Application that Integrates Images, Sound, and Video in a simple User Interface

Aim:

To develop a simple multimedia application that integrates images, audio, video into a unified user interface using python and suitable libraries.

Program:

```
import tkinter as tk
from PIL import Image, ImageTk
from playsound import playsound
import cv2, threading
def image():
    im = Image.open("sample_image.jpg").resize((300,300))
    x = ImageTk.PhotoImage(im)
    panel.config(image=x)
    panel.image = x
def audio():
    playsound("sample_audio.mp3")
def video():
    cap = cv2.VideoCapture("sample_video.mp4")
    while True:
        ret, f = cap.read()
        if not ret: break
        cv2.imshow("Video", f)
        if cv2.waitKey(25) == ord('q'): break
    cap.release()
    cv2.destroyAllWindows()
wi = tk.Tk()
wi.title("Multimedia App")
```

```
[illegible]
[illegible]
[illegible]
[illegible]
[illegible]
[illegible]
[illegible]
[illegible]
```

Exercise:

A multimedia application that accurately counts that integrates
image, audio, and video functionality into single interactive
interface.

Ex.No.:
Date:

Capture Video/Audio from Webcam or Microphone and Display on Multimedia Interface

Aim:
To develop a program that captures live video from the webcam and live audio from the microphone and displays / records them using a simple multimedia interface.

Program:

```
import cv2
import sounddevice as sd
from scipy.io.wavfile import write
import threading
import tkinter as tk
def audio():
    fs = 44100
    x = sd.rec(5*fs, samplerate=fs, channels=2)
    sd.wait()
    write("audio.wav", fs, x)
def video():
    cap = cv2.VideoCapture(0)
    while True:
        r, f = cap.read()
        if not r: break
        cv2.imshow("Webcam", f)
        if cv2.waitKey(1) == ord('q'): break
    cap.release()
    cv2.destroyAllWindows()
w = tk.Tk()
```

```
tk.Button(ui, text="Record Audio",
          command = lambda: threading.Thread(target=audio)
                            .start()).pack()
tk.Button(ui, text="Start Webcam",
          command = lambda: threading.Thread(target=video)
                            .start()).pack()
ui.mainloop()
```

Result:

Live video and audio were successfully captured using webcam and microphone and displayed/saved through a multimedia interface.

Create an application that allows users to send and receive multimedia messages

Aim:
To develop a basic chat-like application that allows sending and receiving text, images, and audio using local file selection and simulated messaging UI.

Program:

```
import tkinter as tk
from tkinter import filedialog, scrolledtext
from PIL import Image, ImageTk
from playsound import playsound
import threading

def text():
    m = entry.get()
    if m:
        chat.insert(tk.END, "You: " + m + "\n")
        entry.delete(0, tk.END)
def Fimage():
p = filedialog.askopenfilename(filetypes = [("Image",
                                "*.jpg *.png *.jpeg)])
    if p:
        chat.insert(tk.END, "Image: " + p + "\n")
        im = Image.open(p)
        im.thumbnail((100, 100))
        x = ImageTk.PhotoImage(im)
        lbl = tk.Label(w, image = x)
        lbl.image = x
        lbl.pack()
```

```
def audio():
    p = filedialog.askopenfilename(filetypes=[("Audio",
                                   "*.mp3 *.wav")])
    if p:
        chat.insert(tk.END,"Audio:" + p + "\n")
    threading.Thread(target=lambda: playsound(p))
                                   .start()

ui = tk.Tk()
ui.title("Messaging App")
chat = scrolledtext.ScrolledText(ui, width=40, height
                                   =15)
chat.pack()
entry = tk.Entry(ui)
entry.pack()
tk.Button(ui, text="Send Text", command=text)
                                   .pack()
tk.Button(ui, text="Send Image", command=image)
                                   .pack()
tk.Button(ui, text="Send Audio", command=audio)
                                   .pack()

ui.mainloop()
```

Result:

A multimedia messaging interface was successfully created with options to send text, image and audio content.